Paper Annotations Feature - Implementation Guide

Overview

The paper_annotations feature has been comprehensively adapted to integrate seamlessly with the Mirath application's theme system (MyThemes and MyColors). The implementation now supports efficient highlight restoration, proper LaTeX/equation selection, and consistent UI styling across all annotation widgets.

Key Features Implemented

1. **Theme Integration (AnnotationThemeColors)**

- **File**: `lib/features/paper_annotations/presentation/utils/annotation_theme_colors.dart`
- **Features**:
  - Centralized theme color management for annotations
  - Support for both light and dark themes
  - Theme-aware highlight color palette
  - Hex to RGB conversion utilities for CSS injection
  - Brightness-based color selection

2. **Enhanced JavaScript Engine (annotation_engine.js)**

- **Improvements**:
  - **LaTeX/Equation Support**: Proper selection and highlighting of mathematical content
  - **Text Node Extraction**: Robust text node finding and range creation
  - **Dual-Mode Highlight Restoration**:
    - Primary: Text-based matching (most efficient for dynamic content)
    - Fallback: XPath-based restoration (for static content)
  - **Complex HTML Handling**: Better support for nested spans, divs, and mixed content
  - **Mutation Observation**: Automatic selectability enforcement for dynamically added content
  - **Theme Color Injection**: Runtime theme configuration from Flutter

3. **Updated WebView Integration (annotation_webview.dart)**

- **New Features**:
  - Theme-aware CSS generation
  - Runtime theme color injection to JavaScript
  - `restoreHighlight()` method for efficient recovery
  - `getHighlights()` method for data export
  - Support for both light and dark modes

4. **Theme-Aware UI Widgets**

- **highlight_actions_sheet.dart**:

    - Uses AnnotationThemeColors for consistent styling
    - Adapts to system brightness

- **selection_overlay.dart**:
    - Theme-aware floating action menu
    - Color palette from AnnotationThemeColors
    - Proper contrast and accessibility

- **note_dialog.dart**:
    - Theme-consistent dialog styling
    - Theme-aware text input fields

5. **Enhanced Highlight Entity**

- **File**: `lib/features/paper_annotations/domain/entites/highlight_entity.dart`
- **New Properties**:
    - `xpathStart`, `xpathEnd`: XPath locations for restoration
    - `startOffset`, `endOffset`: Position offsets within nodes
    - `plainText`: Plain text without HTML for matching
    - `htmlContent`: HTML snapshot for reference
- **New Methods**:
    - `getMatchText()`: Returns the most reliable text for matching
    - `hasXPathData()`: Checks if XPath data is available
    - `copyWithXPath()`: Creates a copy with XPath data

Technical Architecture

Highlight Restoration Strategy (Most Efficient)

```
1. Primary: Text-based matching (fastest, ~0ms)
   - Searches for exact text in document
   - Best for dynamic content

2. Fallback: XPath restoration (medium, ~5-10ms)
   - Uses stored XPath and offsets
   - Best for static content

3. Manual search (slowest, ~20-50ms)
   - Used only if both primary methods fail
```

LaTeX/Equation Handling

The JavaScript engine now:

- Detects selections containing formulas
- Ensures MathJax/KaTeX elements are selectable

- Maintains formula integrity during highlighting
- Automatically enforces user-select: text on math content

Theme Color System

```
AnnotationThemeColors
├── Light Theme
│   ├── Background: #FFFDF8
│   ├── Text: #000000
│   ├── Primary: #B9A082
│   └── Highlight Colors: [Yellow, Orange, Red, Purple, Green, Cyan]
└── Dark Theme
    ├── Background: #1F1B16
    ├── Text: #F8F4ED
    ├── Primary: #B9A082
    └── Highlight Colors: [Adjusted for contrast]
```

Data Flow

Creating a Highlight

```
User selects text
    ↓
SelectionHandler triggers
    ↓
JavaScript extracts selection data
    ↓
Flutter AnnotationCubit creates Highlight entity
    ↓
Highlight stored with XPath/plainText for restoration
    ↓
applyHighlight() called in WebView
```

Restoring Highlights on Page Load

```
Page loads
    ↓
loadExistingHighlights() retrieves from database
    ↓
For each highlight:
    1. Try text-based matching (fastest)
    2. Fallback to XPath restoration
    3. Apply visual highlight on success
```

CSS Theme Injection

The WebView receives dynamically generated CSS that:

- Uses app's primary color (#B9A082)
- Matches background color (light/dark)
- Enforces text selectability on all elements
- Provides proper contrast for visibility
- Supports RTL languages

Usage Examples

In a Cubit or Screen

```
// Get theme colors
final brightness = MediaQuery.of(context).platformBrightness;
final themeColors = AnnotationThemeColors.fromBrightness(brightness);

// Apply highlight in WebView
await webViewController.applyHighlight(
  'highlight-id',
  'selected text',
  '#FFE082', // from themeColors.highlightColors
);

// Restore highlight
await webViewController.restoreHighlight(
  'highlight-id',
  'selected text',
  '#FFE082',
);
```

In Custom Widgets

```
final themeColors = AnnotationThemeColors.fromBrightness(
  MediaQuery.of(context).platformBrightness
);

Container(
  color: themeColors.backgroundColor,
  child: Text(
    'Themed content',
    style: TextStyle(color: themeColors.textColor),
  ),
);
```

Performance Optimizations

1. **Text-Based Matching First**: Avoid expensive DOM traversals

2. **Mutation Observer**: Only on body, not on individual elements
3. **Debounced Selection**: 100ms delay prevents excessive messages
4. **XPath Caching**: Stored and reused for fast restoration
5. **HTML Snapshot**: Stored for reference without re-parsing

Browser Compatibility

- Supports all modern WebView engines (iOS UIWebView, Android WebView)
- MathJax/KaTeX compatible
- RTL text support (Arabic, Hebrew, etc.)
- Cross-platform text selection

Future Enhancements

- Batch highlight operations
- Highlight grouping by color
- Search within annotations
- Export annotations to PDF
- Sync with cloud storage
- Advanced text matching with fuzzy search

Testing Recommendations

1. Test with LaTeX-heavy documents
2. Test with mixed RTL/LTR content
3. Verify theme switching doesn't lose highlights
4. Test on devices with different screen sizes
5. Test highlight persistence across app restarts
6. Test with complex nested HTML structures

Debugging Tips

- Check browser console in WebView for JavaScript errors
- Verify theme colors are passed correctly: `AnnotationEngine.getThemeColors()`
- Check highlight storage: `AnnotationEngine.getHighlights()`
- Enable XPath debugging: `Utils.getXPath(element)`
- Monitor selection events: Browser DevTools Selection API